

ML Engineering for Detection, Classification, Tracking, and Fusion in Counter-UAS Defensive Systems

A research monograph for the Counter-Swarm Defence programme

Author: Victor.I

Role: ML Engineering

Document type: structured research monograph (Pass 1 delivery; expandable)

Defensive scope: Sensing, detection, tracking, fusion, calibration, and operator-facing decision support only. This document does **not** describe weapon guidance, fire-control loops, or electronic-attack execution.

Abstract

Counter-unmanned aircraft system (counter-UAS, C-UAS) defensive software must turn noisy, multi-modal sensor streams into temporally coherent tracks and calibrated risk evidence that operators can trust under time pressure. Machine learning engineering in this setting is not “train a detector and ship.” It is the disciplined composition of classical estimation and data association, modern deep detectors where imagery or spectrograms justify them, multi-object tracking (MOT) evaluation, multi-sensor fusion, continuous calibration and drift monitoring, and a versioned inference path that can be audited, canaried, and rolled back.

This monograph develops that composition for the Counter-Swarm Defence platform. It treats Kalman filtering and its nonlinear relatives as the default kinematic backbone; reviews probabilistic data association (PDAF), joint probabilistic data association (JPDAF), and multiple hypothesis tracking (MHT) in the Bar-Shalom tradition; situates YOLO-style single-shot detectors (Redmon and collaborators) as one EO/IR detection family among others; and binds tracking quality to MOTChallenge-style metrics such as MOTA and IDF1. It then addresses what production systems actually fail at: identity over-merge, domain shift between synthetic twin data and field sensors, miscalibration that inflates operator confidence, adversarial and spoof-like inputs evaluated defensively, and the operational split between NVIDIA Jetson-class edge inference and central x86 fusion.

The intended reader is an ML engineer who must own model cards, training pipelines, registries, and the Counter-Swarm inference-gateway contract—not a researcher seeking novelty for its own sake. Claims are anchored to landmark references. Where the literature is contested or site-specific, the text states assumptions explicitly rather than inventing certainty.

Keywords: counter-UAS; multi-object tracking; Kalman filter; PDAF; JPDAF; MHT; YOLO; MOTA; IDF1; sensor fusion; calibration; concept drift; model registry; edge inference; Jetson; defensive MLOps.

Table of contents

1. [Problem statement and programme context](#)
 2. [Literature foundations and citation discipline](#)
 3. [Observation model and modality contracts](#)
 4. [Classical estimation: Kalman and extensions](#)
 5. [Data association: PDAF, JPDAF, and MHT](#)
 6. [Deep detectors for EO/IR and related modalities](#)
 7. [Classification and soft evidence](#)
 8. [Multi-object tracking pipelines and MOT metrics](#)
 9. [Multi-sensor fusion engineering](#)
 10. [Training data: synthetic twins and real collections](#)
 11. [Training pipelines, registries, and promotion](#)
 12. [Calibration, uncertainty, and drift](#)
 13. [Edge Jetson versus central inference](#)
 14. [Failure modes and adversarial evaluation](#)
 15. [How to prove models in this domain](#)
 16. [Implementation guide: Counter-Swarm inference-gateway](#)
 17. [Non-goals, ethics, and safety boundaries](#)
 18. [Open questions, assumptions, and research agenda](#)
 19. [Conclusion](#)
 20. [References](#)
 21. [Appendix A: Metric definitions](#)
 22. [Appendix B: Model card skeleton](#)
 23. [Appendix C: Association cost design checklist](#)
 24. [Appendix D–K \(worked scenarios through expansion notes\)](#)
 25. [Appendix L: Extended technical deep dive](#)
 26. [Appendix M: Annotated reading path](#)
 27. [Appendix N: Configuration sketch](#)
 28. [Appendix O: Counter-arguments and rebuttals](#)
-

1. Problem statement and programme context

1.1 Restatement of the engineering problem

A defensive counter-swarm software stack receives asynchronous observations from heterogeneous sensors—electro-optical (EO), infrared (IR), radar, radio-frequency (RF) sensing, acoustic, and optionally ADS-B or other cooperative feeds—and must produce:

1. **Detections** (per-frame or per-dwell hypotheses that something of interest is present, with geometry and class soft labels where available).
2. **Tracks** (temporally linked state estimates with identity continuity suitable for operator situation awareness).
3. **Fused entities** (cross-sensor associations that reduce duplicate icons and reconcile conflicting evidence).
4. **Calibrated confidence** (scores that mean something under the site's prior and false-alarm regime, not raw softmax optimism).
5. **Auditable provenance** (which model version, which association hypothesis, which sensor IDs contributed to what the operator saw).

The user pain is not “lack of a neural network.” The pain is **missed small UAS in clutter, ghost tracks, identity flips during crossing trajectories, over-merged swarm members, late fusion under bandwidth constraints, and silent model degradation** after weather, camera firmware, or site layout changes. Meaningful outcomes are reduced time-to-valid-track, reduced false track rate at a declared false-alarm budget, stable identity over mission-relevant horizons, and operator trust that survives an audit.

1.2 Counter-Swarm architectural placement

Within Counter-Swarm Defence, ML engineering owns the path:

```
Raw/sim → labeled datasets (versioned) → training pipeline
          → model registry → inference-gateway → detections/classes
          → metrics / drift monitors → canary / promote / rollback
```

Operational services consume those outputs:

```
observation.v1 → detection → tracking → fusion → behaviour → risk → alert → human
approval
```

The inference-gateway is the serving facade: features or frames in; predictions plus `model_version` out; canary and rollback as first-class operations (see programme architecture notes). Tracking and fusion remain hybrid: classical filters and association logic are preferred for kinematics unless a deep approach is justified by labeled data and proven gains under latency and calibration constraints.

1.3 Ambiguities that must not be papered over

Several quantities are underspecified until site surveys close:

- Exact sensor vendors and proprietary track outputs versus raw measurements.
- Whether EO detection runs at Jetson edge posts or only centrally in early stages.
- Legal and privacy constraints on retaining raw imagery.
- Acceptable false-alarm rates for different alert severities.
- Whether “swarm” is defined by cardinality, coordination features, or both.

Until those close, ML engineering must design **contracts and evaluation harnesses** that remain valid across vendor swaps, and must prefer **central-first inference** with edge packaging as a deploy profile rather than a rewrite.

1.4 Assumptions (challenged)

ID	Assumption	Why it might be wrong	What breaks	Cheap validation
A1	Stage 1 can develop tracking/fusion against a software simulator	Sim fidelity may understate clutter and bias	Field metrics collapse	Hold-out real clips early
A2	Kalman/EKF + JPDA/MHT-class association covers MVP kinematics	Highly agile micro-UAS / strong nonlinearities	Divergent tracks	Residual and NEES monitoring
A3	YOLO-class detectors are adequate for EO when labeled data exists	Tiny distant UAS, IR crossover, glare	Low recall at range	Range-stratified mAP
A4	Hybrid central x86 + Jetson EO is the right compute split	Air-gap or power limits force different topology	Latency/bandwidth miss	Profiled bandwidth tests
A5	Synthetic twin data helps rare swarm regimes	Domain gap dominates	Overconfident models	Twin→real transfer slices

If assumptions outnumber closed facts, the honest posture is **instrumentation and staged promotion**, not early commitment to a single deep end-to-end tracker.

1.5 Smallest version that delivers real value

The smallest ML-valuable slice for Counter-Swarm is:

1. Versioned detector (or sensor-native detection passthrough) behind inference-gateway.
2. Kalman-style single-target filter bank with gated nearest-neighbour or PDA association.
3. Offline MOT-style scoring against labeled sim scenarios (MOTA/IDF1 analogues).
4. Model card + registry entry + forced `model_version` on every prediction event.
5. Drift and calibration monitors on held-out site slices.

Everything else—full MHT, learned association networks, multi-site federated training—comes after that slice is boringly reliable.

2. Literature foundations and citation discipline

2.1 Integrity rule

This programme forbids fabricated DOIs and invented papers. Citations below are landmark works that an examiner would recognise. Secondary surveys and vendor whitepapers may inform engineering practice but do not replace primary references for core algorithms.

2.2 Estimation and tracking canon

Kalman (1960) introduced the recursive linear minimum mean-square estimator that remains the computational workhorse of tracking systems when dynamics and measurement models are approximately linear-Gaussian. The filter’s prediction–update structure, Joseph-form covariance updates for numerical care, and the innovation sequence as a diagnostic are still the first tools an ML engineer should instrument before reaching for deep trackers.

Bar-Shalom and Fortmann (1988), Tracking and Data Association, is the classical monograph for the measurement-to-track association problem: gating, nearest neighbour, probabilistic data association (PDA), and the conceptual frame for clutter and missed detections. **Bar-Shalom, Li, and Kirubarajan (2001), Estimation with Applications to Tracking and Navigation**, extends the practitioner’s toolkit: nonlinear filtering (EKF, UKF treatments in the broader literature), interacting multiple model (IMM) ideas, and multi-target association in navigation-relevant settings. Together these texts justify why Counter-Swarm treats association as a first-class probabilistic decision problem rather than a post-hoc clustering cosmetic.

Multiple hypothesis tracking (MHT) in the Bar-Shalom ecosystem builds on the insight that deferred association decisions—maintaining a tree of hypotheses—can outperform greedy assignment when measurements are ambiguous. Reid’s earlier algorithmic framing of multi-target tracking and subsequent practical MHT implementations (track-oriented MHT variants in industry) matter operationally: hypothesis management cost grows quickly, so engineering limits (N-scan pruning, clustering, gating) dominate theory.

2.3 Detection deep learning

Redmon, Divvala, Girshick, and Farhadi (2016), You Only Look Once (YOLO), reframed object detection as a single regression problem over a spatial grid, trading some localisation precision for speed—directly relevant to edge EO pipelines. **Redmon and Farhadi (2017), YOLO9000**, and **Redmon and Farhadi (2018), YOLOv3**, iterated architecture, multi-scale prediction, and training practice. Later YOLO-family forks (community and commercial) exist; this monograph cites the Redmon-era papers as the lineage justification for “YOLO-style” single-shot detectors in Counter-Swarm model cards, while requiring each deployed artifact to name its exact architecture, weights provenance, and licence.

Two-stage detectors (Faster R-CNN lineage) and transformer detectors remain valid alternatives when accuracy at range outweighs latency. The engineering rule is comparative evaluation under site latency budgets, not fashion.

2.4 MOT evaluation literature

Multi-object tracking quality is not reducible to detection mAP. **Bernardin and Stiefelhagen (2008)** introduced the CLEAR MOT metrics, including Multiple Object Tracking Accuracy (MOTA), which combines false positives, false negatives, and identity switches in a single scalar. The **MOTChallenge** benchmarks (see Milan, Leal-Taixé, Reid, Roth, and Schindler, MOT16 and related MOTChallenge reports; Leal-Taixé et al. on MOTChallenge protocols) popularised standardised sequences, identity-aware metrics such as IDF1, and public leaderboards. Counter-Swarm should not pretend outdoor UAS tracking is identical to pedestrian MOT, but it **should** reuse the metric definitions and the discipline of identity-aware scoring.

2.5 Calibration and modern network overconfidence

Detection and classification scores are often poorly calibrated. Guo, Pleiss, Sun, and Weinberger (2017), “On Calibration of Modern Neural Networks,” is a widely cited demonstration that deeper networks can be systematically overconfident; temperature scaling and related post-hoc methods are practical first remedies. Expected Calibration Error (ECE) and reliability diagrams belong on Counter-Swarm dashboards whenever soft scores influence operator-visible risk.

2.6 What literature does not give you

Landmark papers do not specify your clutter density, your camera MTF, your legal retention window, or your false-alarm budget at a given site. They also do not authorise skipping association theory because a detector “looks good” on a demo reel. The remainder of this monograph converts literature into contracts, metrics, and failure analysis for Counter-Swarm.

3. Observation model and modality contracts

3.1 Canonical observation as the stability hinge

Counter-Swarm normalises vendor and simulator inputs into `observation.v1` (conceptual schema in programme schemas/). ML components must consume validated observations or derived tensors, not ad-hoc vendor SDK types, so that training and serving share one semantic layer.

Minimum fields an ML engineer should treat as sacred:

- Sensor identity and modality.
- Sensor timestamp (`timestamp_utc`) and platform receive time (`received_at_utc`).
- Geometry in a declared CRS (or image-plane coordinates with camera model reference).
- Quality flags (including clock skew, saturation, RF interference indicators where available).
- Payload references for heavy blobs (frames, spectrograms) in object storage.

Fusion and tracking must tolerate out-of-order events within a watermark; training pipelines must store enough provenance to reproduce a labeled example from raw artifacts.

3.2 Modality-specific measurement models

Modality	Typical measurement	Hard problems for ML/tracking
EO	Bounding boxes / keypoints in image plane	Scale, glare, motion blur, tiny targets
IR	Same, different contrast physics	Cross-over temperatures, weather
Radar	Range, bearing, Doppler, RCS-like features	Clutter, multipath, limited class info
RF sensing	Direction, signal features, emitter class soft labels	Spoofing, dense spectrum
Acoustic	Bearing, signature class	Wind noise, short range
Cooperative	Reported state	Trust, spoof, intermittency

Deep models usually enter strongest on EO/IR and RF spectrogram classification. Radar tracking often remains classical with learned clutter/classifier assists. Do not force a single network across modalities without a fusion design that respects each measurement's error model.

3.3 Detection as a random finite set intuition

Even if Counter-Swarm does not implement full Random Finite Set (RFS) filters (PHD/CPHD/GLMB) in Stage 1, engineers should think in RFS terms: the number of targets is unknown; detections include clutter; targets are missed with probability $(P_D < 1)$. That intuition prevents naive “every box is a target” pipelines and motivates PDA/JPDA/MHT or principled assignment costs.

4. Classical estimation: Kalman and extensions

4.1 Why classical filters remain first-class in C-UAS ML engineering

Deep networks excel at turning pixels or spectrograms into measurements. They are comparatively weak, unless heavily specialised and data-rich, at enforcing kinematic consistency, providing well-understood covariance for gating, and failing loudly when the process model is violated. Kalman filtering, properly engineered, gives:

- A recursive state and covariance that association algorithms can gate against.
- Innovation whiteness tests and Normalised Estimation Error Squared (NEES) / Normalised Innovation Squared (NIS) diagnostics when ground truth or consistency checks exist.
- Predictable compute suitable for central track updates under NFR latency budgets.

Kalman (1960) introduced the recursive linear minimum mean-square estimator that remains the computational workhorse when dynamics and measurement models are approximately linear-Gaussian. The filter prediction-update structure, careful covariance updates, and the innovation sequence as a diagnostic are still the first tools an ML engineer should instrument before reaching for deep trackers.

In discrete time, with state (x_k) and measurement (z_k) :

[

$$x_k = F_{k-1} x_{k-1} + w_{k-1}, \quad z_k = H_k x_k + v_k$$

]

with $(w \sim \mathcal{N}(0, Q))$, $(v \sim \mathcal{N}(0, R))$. The engineering burden is choosing (F) , (Q) , (H) , (R) that match the sensor and platform, not re-deriving the algebra.

4.2 Motion models for small UAS

Common nearly-constant velocity (NCV) and nearly-constant acceleration (NCA) models are starting points. Small UAS can hover, accelerate aggressively, and change heading quickly relative to ground vehicles. Therefore:

1. Prefer IMM (interacting multiple model) mixtures—for example NCV plus a manoeuvring model—when a single (Q) cannot cover both cruise and agile segments. IMM treatments appear throughout the Bar-Shalom tracking literature (Bar-Shalom, Li, and Kirubarajan, 2001).
2. Inflate (Q) adaptively when NIS is persistently large, but log the adaptation; silent (Q) hacking destroys auditability.
3. Separate filter tuning by modality: radar range-bearing filters differ from image-plane trackers that later project via camera models.

4.3 Nonlinear filters

Extended Kalman filters (EKF) linearise nonlinear measurement functions (for example converting Cartesian state to range-bearing). Unscented Kalman filters (UKF) and particle filters address stronger nonlinearities at higher cost. For Counter-Swarm Stage 1-2, EKF/UKF on radar and a linear or affine image-plane tracker for EO, with explicit camera calibration versioning, is usually enough. Reach for particle filters only with a clear residual failure mode that EKF/UKF cannot fix under load.

4.4 Filter banks and track lifecycle

Operational tracking is a manager around filters:

- **Birth:** unassociated detections initiate tentative tracks after M-of-N confirmation rules.
- **Confirm:** promote to confirmed when evidence accumulates (score or hit history).
- **Coast:** predict without update through brief misses (within coast timeout).

- **Death:** delete on prolonged misses or low existence probability.
- **Merge/split:** dangerous operations—see failure modes on over-merge.

Every lifecycle transition should emit an auditable event with track ID, filter configuration version, and contributing observation IDs.

4.5 Consistency monitoring as MLOps for classical models

Treat filter configuration as a model: version it, regression-test it on scenario packs, and monitor NIS distributions in production. A deep detector upgrade that changes measurement noise statistics without retuning $\backslash(R\backslash)$ is a classic silent regression. Coupling detector model cards to tracker configuration versions is mandatory.

4.6 Gains and losses of a Kalman-first posture

Gains	Losses
Interpretable uncertainty for gating and fusion	Weak on raw pixels without a detector front-end
Mature association literature	Poor if dynamics are badly misspecified
Deterministic latency envelopes	Not a substitute for appearance-based re-ID in heavy occlusion
Easy to audit	Can look underwhelming next to demo-grade deep trackers on curated video

Recommendation: Keep Kalman/EKF/IMM as the kinematic spine. Add learned components at measurement formation and optional appearance embeddings for association—not as an opaque replacement for state estimation in MVP.

4.7 Numerical and implementation hygiene

Production filters fail for mundane reasons: non-symmetric covariances after careless updates, units mismatches (degrees vs radians), and frame transforms applied twice. Require:

- Unit tests on synthetic linear-Gaussian cases with NEES coverage checks.
- Single authoritative transform library for CRS and camera models.
- Explicit rejection of non-finite innovations.
- Configuration schemas validated at service boot.

These are not glamorous, but they dominate field defect rates relative to exotic filter variants.

5. Data association: PDAF, JPDAF, and MHT

5.1 The association problem

Given predicted tracks and a set of measurements at time (k) , decide which measurements belong to which tracks, which are clutter, and which tracks are missed. Errors here dominate operator-visible failures: identity switches, fragmentation, and ghost tracks.

Gating (validation regions) using innovation covariance is the first computational cut. Association then chooses among:

- Global nearest neighbour (GNN) / Hungarian assignment on a cost matrix.
- Probabilistic Data Association (PDA) for single targets in clutter.
- Joint Probabilistic Data Association (JPDA) for multiple targets sharing measurement origin uncertainty.
- Multiple Hypothesis Tracking (MHT) deferred decisions.

Bar-Shalom and Fortmann (1988), *Tracking and Data Association*, remains the conceptual home for PDA and the cluttered measurement model. Bar-Shalom, Li, and Kirubarajan (2001), *Estimation with Applications to Tracking and Navigation*, extends multi-target and navigation-relevant estimation practice that ML engineers inherit when they own tracker configuration as a product artifact.

5.2 PDAF (Probabilistic Data Association Filter)

PDA softens hard assignment: for a single track, all gated measurements contribute to the update weighted by association probabilities that account for clutter density and detection probability (P_D) . This reduces brittle wrong associations in moderate clutter at the cost of covariances that can become optimistic if clutter models are wrong.

Engineering notes:

- Estimate clutter density carefully (parametric Poisson clutter volume assumptions versus empirical spatial rates).
- PDA is per-track; it does not fully resolve competition between closely spaced targets—hence JPDA.
- Log effective association entropy; persistently flat weights mean the filter is guessing.

5.3 JPDAF (Joint Probabilistic Data Association Filter)

JPDA considers joint association events across a cluster of targets and measurements, marginalising to per-track association weights. It is the classical answer to crossing tracks and small formations—the swarm-relevant case. Computational cost grows with cluster size; practical systems cluster tracks that share gated measurements and solve JPDA within clusters.

Failure mode specific to C-UAS: over-smoothing of closely spaced swarm members can pull states together (coalescence). Mitigations include tighter gates when class or appearance features separate targets, careful use of repulsive or minimum-separation heuristics only when

physically justified, and escalating to MHT when JPDA weights remain ambiguous over several scans.

5.4 MHT (Multiple Hypothesis Tracking)

MHT maintains alternative association histories, pruning with N-scan windows, hypothesis scores, and clustering. In the Bar-Shalom ecosystem, deferred decisions outperform greedy assignment when early hard mistakes are expensive—exactly the identity-stability requirement for operator tracks and later behaviour features. Reid’s multi-target tracking formulation is historical background; engineering MHT is dominated by hypothesis management cost.

Tradeoffs:

Approach	When it wins	When it becomes a liability
GNN	Sparse, well-separated targets	Crossing / dense clutter
PDA	Single target, cluttered	Multi-target contention
JPDA	Small clusters, real-time	Large unresolved clusters; coalescence
MHT	Ambiguous scenes, identity critical	CPU/memory; tuning complexity

5.5 Hybrid modern practice

Many production systems combine:

1. Deep detector measurements.
2. Motion-gated cost matrix augmented with appearance cosine distance.
3. JPDA or auction/Hungarian for real-time primary hypothesis.
4. Limited MHT or tracklet stitching offline or asynchronously for identity repair.

Learned association (graph neural networks, Transformer trackers) can be evaluated as challengers, but must beat JPDA/MHT baselines on IDF1 and switch rate under the same detection inputs and latency caps.

5.6 Association costs for Counter-Swarm

A practical cost vector includes:

- Mahalanobis innovation distance.
- Class compatibility (soft).
- Optional appearance embedding distance for EO.
- Sensor credibility weights in fused space.
- Time-since-last-hit penalties for coasting tracks.

Document the cost formula in the tracker configuration version. Changing costs without version bumps is a governance defect.

5.7 Worked conceptual example: two-UAS cross

Two confirmed tracks approach a crossing. Detector returns two boxes with moderate scores plus one clutter box from a bird.

1. Gate: clutter may fall inside one gate depending on $\backslash(R\backslash)$ and $\backslash(Q\backslash)$.
2. GNN may assign greedily and flip identities after the cross.
3. JPDA spreads weight across feasible joint events, delaying hard identity commitment in the state update while still producing a displayable primary hypothesis.
4. MHT retains both identity continuations for N scans; if subsequent appearance or RF evidence supports one hypothesis, the alternative is pruned.

The ML engineering lesson: detector mAP improvements that add duplicate boxes without association upgrades can **worsen** IDF1. Measure both.

6. Deep detectors for EO/IR and related modalities

6.1 Role of detectors in the pipeline

Detectors convert images (or other dense tensors) into measurement sets for association. They do not replace tracking. YOLO-style detectors are attractive for Counter-Swarm because single-shot architectures map cleanly onto TensorRT deployment on Jetson-class hardware and onto central GPU batches.

6.2 YOLO lineage (Redmon et al.)

Redmon, Divvala, Girshick, and Farhadi (2016) introduced YOLO (*You Only Look Once*) as a unified real-time detector that frames detection as spatial regression over a grid. Redmon and Farhadi (2017), YOLO9000, and Redmon and Farhadi (2018), YOLOv3, iterated multi-scale prediction and training practice. For model cards, cite the architectural generation actually used and the training codebase. Do not claim “YOLO” without version; inference performance and failure modes differ across generations and forks.

Design implications:

- Grid and anchor assignment affect small-object recall—critical for distant UAS.
- Input resolution versus latency is a first-order tradeoff on Jetson Orin Nano/NX.
- Non-maximum suppression (NMS) thresholds interact with swarm density: aggressive NMS causes under-count; weak NMS floods the tracker with duplicates.

6.3 Training objectives and small-object regimes

Standard COCO-style training is a poor prior for tiny airborne targets. Practices that matter:

- Resolution pyramids and tiling for high-megapixel cameras (with stitch-aware duplicate suppression).

- Copy-paste and mosaic augmentations tuned so synthetic composites do not create impossible illumination (domain gap risk).
- Range-stratified sampling: force enough far-range positives in each batch.
- Hard negative mining on birds, balloons, helicopters, clouds, and lens flare.

6.4 Beyond YOLO

Evaluate alternatives when metrics demand:

- Two-stage detectors for accuracy-critical central offline review tools.
- Transformer detectors if latency budget allows and small-object gains are proven.
- Classical CFAR-like front ends for radar, with optional learned classifiers on crops or snippets.

6.5 Detector outputs as probabilistic measurements

Map detector confidence to measurement noise $\sigma(R)$ only after calibration. A raw score of 0.9 is not $\sigma(P_D=0.9)$ and is not a licence for tiny $\sigma(R)$. Prefer empirical localisation error versus score bins on validation data.

6.6 IR-specific notes

IR detectors need separate training sets; EO weights rarely transfer cleanly across thermal crossover conditions. Maintain distinct model registry entries and promotion criteria per modality, even if architectures match.

6.7 Throughput engineering

Batch size, FP16/INT8 quantisation, and preprocessing CPU contention determine whether a detector meets p95 latency. Quantisation must be validated on the same stratified slices as full precision; small-object recall often dies first under aggressive INT8. Jetson TensorRT engines are hardware-and-JetPack specific—treat engine builds as first-class artifacts in the registry, not as transparent caches.

7. Classification and soft evidence

7.1 What classification means here

Classification may attach labels such as `uas_multicopter`, `uas_fixed_wing`, `bird`, `helicopter`, `unknown`, `clutter` to detections or tracks. Soft evidence should flow into risk as calibrated probabilities, not one-hot fantasies.

7.2 Where classifiers sit

1. Per-detection head on the detector (multi-class YOLO-style).

2. Secondary classifier on cropped chips (higher capacity, extra latency).
3. Track-level classifier aggregating evidence over time (HMM or temporal net).
4. RF/acoustic specialist models producing class soft labels fused later.

Track-level aggregation usually stabilises flicker. It must not silently invent class changes without logging.

7.3 Open-set and reject options

Operational sites see novel airframes. Force an unknown / reject class with a dedicated evaluation slice. Closed-set accuracy without reject quality is misleading for C-UAS.

7.4 Hierarchy and policy

Risk engines may care more about “threat-relevant airborne object” than fine subtype. Align label taxonomy with product risk policy early; relabeling later invalidates historical metrics.

7.5 RF classification sketch

RF pipelines often start with classical features (bandwidth, hopping patterns, protocol fingerprints where legally and technically available) plus shallow models, escalating to CNN/RNN/Transformer classifiers on spectrograms when corpora justify them. Keep RF models behind the same inference-gateway contracts so canary and rollback match EO practice.

8. Multi-object tracking pipelines and MOT metrics

8.1 End-to-end tracking pipeline (recommended MVP shape)

```
Frames/obs → Detector (versioned) → Measurement set
  → Gating → Association (GNN/JPDA/MHT-lite)
  → Kalman/IMM update → Track manager
  → Optional appearance re-ID features
  → Track events on bus (with model_version + tracker_config_version)
```

Do not ship opaque deep-SORT-only stacks without classical baselines and MOT metrics on Counter-Swarm scenario packs.

8.2 MOTA and identity metrics

Bernardin and Stiefelhagen (2008) introduced the CLEAR MOT metrics, including Multiple Object Tracking Accuracy (MOTA), which combines false positives, false negatives, and identity switches relative to ground-truth object counts over time. The MOTChallenge benchmarks—see Milan, Leal-Taixé, Reid, Roth, and Schindler (MOT16 and related reports) and the

MOTChallenge evaluation protocols associated with Leal-Taixé, Bernardin/Stiefelhagen-derived CLEAR metrics, and subsequent challenge editions—popularised standardised sequences and identity-aware metrics such as IDF1.

For Counter-Swarm:

- Report MOTA, IDF1, identity switch rate, fragmentation, and mostly-tracked / mostly-lost analogues.
- Stratify by range, clutter level, swarm cardinality, modality, and day/night.
- Never optimise MOTA alone: a system can game detection at the expense of identity.

Formal MOTA (CLEAR) has the schematic form:

\[

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}$$

\]

IDF1 emphasises how long predicted identities correctly cover ground-truth identities. Both require a matching threshold: IoU in image space or Mahalanobis/distance gates in world coordinates. Freeze the matching rule in CI.

8.3 Mapping MOTChallenge discipline to UAS

Pedestrian MOT datasets differ in motion scale, camera motion, and class confusers. Reuse:

- Clear matching thresholds.
- Public evaluation scripts frozen in CI.
- Separation of private test scenarios.

Build programme-owned scenario packs from the digital twin and from consented field captures. Cite MOTChallenge as methodological precedent, not as a claim that public MOT rankings transfer to C-UAS.

8.4 Online versus offline tracking

Operators need online tracks. Offline MHT or tracklet stitching can improve after-action review and training labels. Keep online/offline configs distinct in the registry; never silently swap offline-quality identity repair into the live path without latency proof.

8.5 Tracker evaluation contracts

Each tracker release must publish:

1. Detection input freeze (which detector version produced measurements).
2. Association algorithm and cost formula version.
3. Scenario pack IDs and metric tables.
4. Latency p50/p95 for track update on reference hardware.

5. Known failure slices (for example, cardinality at or above 12 within 50 m: IDSW elevated).

8.6 Appearance embeddings and re-identification

Short-horizon kinematic association fails under occlusion and camera handoff. Appearance embeddings (trained with triplet or contrastive losses on UAS chips) can reduce IDSW. Risks:

- Embeddings trained on one site's paint/airframes fail on novel craft (open-set).
- Lighting and IR crossover destroy cosine margins.
- Embedding extractors add latency on Jetson.

Treat re-ID models as optional association features with their own model cards, not as identity oracles.

8.7 Track quality scores for operators

Expose a track quality vector: hit ratio, coast age, association entropy, class entropy, fused modality count. Risk and UI layers consume these; ML owns their definitions and calibration against "operator would trust this track" labels where available.

9. Multi-sensor fusion engineering

9.1 Fusion goals

Fusion exists to:

- Reduce duplicate operator icons for the same physical object.
- Improve kinematic quality by combining complementary measurements.
- Maintain evidence links for audit (this fused track cites radar track R7 and EO track E3).

Fusion is not an excuse to hide modality failures. If IR is blind in crossover, the fused track must carry quality degradation flags.

9.2 Architectural approaches (explore before converging)

Approach A — Track-to-track fusion (central)

Each modality runs local tracking; central fusion associates tracks and merges states.

- Gains: modality teams decouple; edge can send tracks not video.
- Losses: information loss versus measurement-level fusion; correlated errors if local filters share process noise naively.
- Liability: double-counting covariance if cross-covariance is ignored.

Approach B — Measurement-level central fusion

Raw or normalised measurements associate into a single track filter.

- Gains: statistically cleaner when models are right.
- Losses: bandwidth (especially EO frames); tighter clock sync needs.
- Liability: central bottleneck; harder edge autonomy if links drop.

Approach C — Hybrid (recommended baseline)

Edge EO detects (and optionally light-tracks) on Jetson; radar/RF adapters emit measurements or local tracks; central fusion performs cross-sensor association with explicit sensor bias states.

- Gains: matches Counter-Swarm hybrid compute guidance; bandwidth-aware.
- Losses: more moving parts; version skew between edge and central.
- Liability: becomes unmaintainable if schemas drift.

Recommendation: Approach C for Counter-Swarm, central-primary in early stages (even EO inference centralised until cameras force edge). Design schemas so promoting EO detect to Jetson does not rewrite fusion.

9.3 Cross-sensor association

Costs differ from single-sensor MOT: different state spaces, biases, and latencies. Maintain:

- Sensor alignment parameters as versioned calibration artifacts.
- Time registration with explicit skew flags.
- Soft class agreement terms.
- Gates large enough for residual misregistration—then fix calibration rather than eternally widening gates.

9.4 Out-of-order and late data

Use watermarks and bounded retroactive updates. If a late EO detection arrives, either (a) update within N-scan if hypothesis state is retained, or (b) attach as evidence without rewriting the displayed state if latency SLA forbids. Product must choose; ML must implement the chosen semantics with tests.

9.5 Fusion outputs

Fused tracks should expose:

- Kinematic state and covariance.
- Contributing track/observation IDs.
- Per-modality freshness.
- Calibrated existence and class probabilities.
- `fusion_config_version` and `contributing_model_version` set.

9.6 Decorrelation and naive fusion hazards

A common defect is fusing two tracks that already incorporated the same process model or the same upstream detection, yielding overconfident covariances. Mitigations: measurement-level fusion where possible; track-to-track fusion methods that account for cross-covariance; or

conservative covariance inflation with explicit labeling as approximate. Overconfidence is a safety defect in operator systems even when kinematics look smooth.

9.7 Behaviour features after fusion

Coordination and anomaly indicators should consume fused tracks with modality quality flags. Feeding behaviour models with flickering pre-fusion identities creates false swarm alarms. ML engineering owns the handoff contract to behaviour services: minimum track age, maximum coast age, and required existence probability.

10. Training data: synthetic twins and real collections

10.1 Why synthetic data is necessary but insufficient

Rare swarm geometries, dangerous proximity to infrastructure, and adverse weather cannot be collected on demand. Counter-Swarm’s digital twin and scenario catalogue should generate labeled observations and optional rendered EO/IR. Synthetic data wins on cardinality and controlled ablations. It loses on sensor nuisances: compression artifacts, autofocus hunting, real clutter statistics, and vendor-specific radar quirks.

10.2 Data mixture policy

A defensible mixture:

Slice	Role	Risk if overused
Real site A day	Primary distribution	Overfit to one site
Real site A night / IR	Modality shift	Small N
Real negatives (birds, empty sky)	FPR control	Label noise
Twin swarm high-N	Stress association	Domain gap
Twin fault injections	Robustness	Unrealistic faults
Adversarial/spoof-like eval (defensive)	Security eval	Training on attacks may not generalise

Promotion gates must include **real-only** holdouts. Twin-only metrics are development indicators, not ship criteria.

10.3 Labeling standards

- Bounding boxes with explicit occlusion and truncation flags.
- Track IDs continuous across occlusions when human-annotator certainty is high; otherwise break IDs rather than invent continuity.
- Class taxonomy versioned; dual-annotator agreement on hard classes.
- Time sync verification when multi-sensor labels are fused.

10.4 Dataset versioning

Store:

- Manifest of raw artifact URIs and hashes.
- Label files with taxonomy version.
- Split definition (by scenario and time—not random frames that leak identity across train/test).
- Licence and retention constraints.
- PII/privacy review status for any incidental imagery.

Training jobs pin dataset versions immutably.

10.5 Leakage failure modes

Common leakages in tracking datasets:

- Adjacent frames from the same track in train and test.
- Twin scenarios that share procedural generation seeds with eval packs.
- Fine-tuning on “test site” clips during demos.

Require scenario-level and time-blocked splits. Publish leakage checks in CI.

10.6 Synthetic rendering tips that reduce domain gap

- Domain randomisation of textures, sky HDRIs, and camera ISP approximations.
- Noise models matched to measured camera noise, not generic Gaussian.
- Shadow and motion-blur consistency with camera exposure settings.
- Occasional compression to match operational encoding.

Still expect a gap; plan fine-tuning and test-time augmentation policies carefully.

10.7 Human annotation workflow

Annotation is a production system: guidelines, spot audits, inter-annotator metrics, and feedback when models repeatedly fail on a pattern. Budget annotation as a first-class line item; underfunded labels produce confident, wrong detectors.

11. Training pipelines, registries, and promotion

11.1 Pipeline stages

A production-grade training pipeline for Counter-Swarm detectors and classifiers:

1. **Ingest:** pull pinned dataset version; verify hashes.
2. **Validate:** schema checks, empty-label rates, class histograms, split leakage tests.
3. **Train:** reproducible seeds, logged hyperparameters, hardware fingerprint.

4. **Evaluate:** stratified mAP/F1, calibration ECE, robustness slices, latency on reference engines.
5. **Package:** ONNX/TensorRT/torchscript artifacts; SBOM; licence scan.
6. **Register:** model registry entry with metrics, dataset pins, card.
7. **Canary:** shadow or limited traffic via inference-gateway.
8. **Promote or rollback:** gated by SLO and human approval for safety-impacting models.

Training is offline and separated from the operational bus by design (programme data-flow rule).

11.2 Model registry requirements

Every operator-visible inference carries `model_version`. The registry must store:

- Semantic version or content-addressed ID.
- Parent lineage (fine-tune base).
- Dataset and code commit SHAs.
- Metrics tables and links to evaluation reports.
- Intended hardware targets (x86 CUDA, Jetson Orin NX TensorRT 8.x, etc.).
- Known limitations and restricted use statements.
- Approval signatures for promotion stages.

Unversioned notebooks are explicitly non-goals for production.

11.3 Experiment tracking versus registry

Experiment trackers (hyperparameter search logs) are not registries. Only candidates that pass evaluation contracts enter the registry. This prevents “best TensorBoard screenshot” from becoming production.

11.4 Reproducibility floor

Exact bitwise reproducibility across GPU architectures is often unrealistic. Require:

- Dataset and code pins.
- Container image digest.
- Reported metric variance across three seeds for primary metrics.
- Ability to re-run evaluation even if training cannot be bitmatched.

11.5 Continuous training triggers

Retrain on:

- Scheduled cadence (for example monthly) if data accrues.
- Drift alarms exceeding thresholds.
- Sensor firmware or ISP changes.
- Taxonomy changes.
- Critical false-negative incidents with labeled postmortems.

Avoid perpetual online learning on the live bus without sandboxing; silent self-training on contaminated labels is a known failure mode.

11.6 Classical tracker configs in the same governance model

JPDA parameters, $\backslash(P_D\backslash)$, clutter rates, and IMM mode sets must live in a configuration registry with the same promotion discipline as neural weights. A YAML edit that doubles process noise is a model change.

11.7 Cost controls

Track GPU hours, storage of raw video, and annotation cost per point of mAP. Reject training programmes that cannot state expected operational value. Prefer smaller models that meet latency and calibration over marginal accuracy gains that force central-only deployment when edge is required.

12. Calibration, uncertainty, and drift

12.1 Why calibration is non-negotiable

Operator risk displays and alert thresholds assume scores behave like probabilities. Modern networks are often overconfident (Guo, Pleiss, Sun, and Weinberger, 2017). In C-UAS, overconfidence produces premature high-severity alerts; underconfidence buries real tracks.

12.2 Metrics

- Reliability diagrams.
- Expected Calibration Error (ECE) and class-wise ECE.
- Brier score for class soft labels.
- For detectors: calibration of objectness versus empirical precision at score thresholds.

12.3 Post-hoc and trained calibration

Temperature scaling on a held-out calibration set is a strong first intervention for classifiers. Platt scaling and isotonic regression are alternatives with overfitting risks on small calibration sets. For detectors, calibrate decision thresholds per site and per range bin; do not assume a universal 0.5 cutoff.

Bayesian deep learning and ensembles improve uncertainty in research settings; adopt only when latency and ops cost allow, and still verify calibration empirically.

12.4 Filter consistency versus neural scores

NIS/NEES address Kalman consistency; ECE addresses neural score calibration. Fusion must not multiply an overconfident class score with an overconfident existence probability without empirical joint calibration on scenario packs.

12.5 Drift typology

Drift type	Example	Monitor
Data drift	New camera, seasonal foliage	Input embedding distances, pixel stats
Concept drift	New airframe types	Class confusion shifts
Label drift	Taxonomy change	Schema version mismatch alarms
Upstream drift	Detector NMS change	Measurement rate spikes
Environmental drift	Fog season	Range-stratified recall drop

12.6 Drift response playbook

1. Alert with slice diagnosis.
2. Freeze promotions.
3. Shadow evaluate last known-good model.
4. Collect targeted labels.
5. Retrain or retune thresholds.
6. Canary carefully.
7. Document incident in model lineage.

12.7 Site-specific thresholding

A model can be globally well-calibrated and still wrong for a site's false-alarm budget. Maintain site overlays: threshold packs versioned separately from weights, promoted with local operations approval.

13. Edge Jetson versus central inference

13.1 Programme baseline

Counter-Swarm systems engineering recommends hybrid compute: central x86 for fusion, risk, console, bus, database, and audit; NVIDIA Jetson Orin-class for EO/IR edge inference when cameras require it; Raspberry Pi only for light adapters—not primary detection. Stage 1 develops on a single x86 lab host with Compose; Jetson is a later deploy profile.

13.2 Decision criteria for placing a model

Criterion	Prefer edge (Jetson)	Prefer central
Bitrate of raw video	High; link constrained	Manageable backhaul
Latency to first detection	Camera-local critical	Fusion wait dominates anyway
Model size / TensorRT support	Fits Orin class	Needs large GPU
Multi-sensor correlation	Local only insufficient	Required
Ops maturity	Team can manage JetPack fleets	Central MLOps stronger
Training	Never on edge field nodes	Always central/offline

13.3 Packaging implications

- Multi-arch builds: `linux/amd64` central, `linux/arm64` Jetson.
- TensorRT engine artifacts keyed by JetPack and GPU SKU.
- Health endpoints reporting engine warm status, thermal throttling, and dropped frames.
- Store-and-forward of detections when the site link drops; central fusion marks degraded mode.

13.4 Version skew

Edge nodes will lag central registry temporarily. Policies:

- Maximum skew windows.
- Incompatible schema versions rejected with loud alarms.
- Canary by node cohort, not all cameras at once.

13.5 What not to put on Pi

YOLO-class EO detection under operational load is not a Pi job in this architecture. Using Pi for teaching adapters is fine; claiming Pi as the vision edge for trials damages credibility and metrics.

13.6 Energy, thermal, and reliability

Jetson nodes throttle. Track infer latency against temperature. A model that meets p95 in the lab at 20 C may miss SLO at 45 C ambient in a sealed enclosure. Include thermal soak in promotion tests for edge profiles.

13.7 Security of edge models

Edge devices are physical attack surfaces. Sign model artifacts, encrypt at rest where policy requires, and assume an adversary may extract weights. Avoid embedding secrets in model packages. Defensive adversarial evaluation (Section 14) still applies.

14. Failure modes and adversarial evaluation

14.1 Ranked operational failure modes

1. **Identity over-merge in swarms:** multiple UAS collapse to one track; cardinality wrong; behaviour “coordination” false.
2. **Fragmentation / over-split:** one UAS becomes many tracks; alert spam.
3. **Domain shift:** twin-trained detector fails on real ISP/weather.
4. **Calibration collapse after quantisation:** INT8 edge engines overconfident or under-recalling.
5. **Association starvation under clutter:** JPDA clusters explode CPU; tracks coast to death.
6. **Clock skew:** fusion associates wrong-time measurements.
7. **Silent registry rollback failure:** canary broken; bad model stuck.
8. **Adversarial or spoof-like inputs:** patches, projected patterns, RF mimicry—evaluated defensively for detection robustness, not for offensive recipes.

14.2 Over-merge: mechanisms and mitigations

Mechanisms: wide gates, aggressive NMS suppressing neighbours, JPDA coalescence, fusion associating distinct modality tracks too eagerly, appearance embeddings that are too invariant.

Mitigations: cardinality-aware evaluation, minimum measurable separation metrics, NMS tuning paired with tracker tests, MHT for ambiguous clusters, dual-hypothesis UI for operators when entropy is high, hard stop on merging confirmed tracks without multi-scan evidence.

14.3 Adversarial evaluation (defensive)

Purpose: measure degradation under malicious or nuisance inputs so operators know failure envelopes. Practices:

- Hold-out sets with glare, lasersat-like bloom (as nuisance), printed distractors, and RF interference patterns from lab generators.
- Score recall/precision under attack slices; never claim absolute robustness.
- Separate **evaluation** artifacts from **training** recipes that would amount to attack instruction.

This monograph intentionally omits step-by-step construction of offensive adversarial attacks or spoof transmitters.

14.4 Birds and biological false positives

A perennial C-UAS nuisance. Mitigations are data and temporal: flight dynamics features, multi-sensor confirmation, and calibrated class probabilities with reject options. Purely visual single-frame classifiers will fail seasonally; budget for that drift.

14.5 Human-automation failure coupling

ML can be “accurate” yet operationally harmful if alert rates exceed operator capacity. Couple ML promotion gates to alert budget simulations from product and human-factors workstreams.

15. How to prove models in this domain

15.1 What “proof” can mean

In safety-adjacent defensive software, proof is not a single mathematical certificate for a deep network. It is a **structured assurance case**: claims, evidence, and arguments that survive audit. ML engineering owns the evidence packs for perception and tracking claims.

15.2 Claim hierarchy (example)

1. **C1:** On scenario pack SP-04 (day EO, cardinality 1-5), detector D meets $\text{recall} \geq R_0$ at $\text{precision} \geq P_0$ at operating threshold T_0 .
2. **C2:** Given detections from D, tracker Tr meets $\text{IDF1} \geq I_0$ and $\text{IDSW rate} \leq S_0$ on SP-04.
3. **C3:** ECE of class scores $\leq E_0$ on calibration set Cal-02.
4. **C4:** p_{95} inference latency $\leq L_0$ on reference hardware H_0 (and Jetson profile J_0 if applicable).
5. **C5:** Under listed nuisance slices, recall degradation remains within bound B_0 and fails safe (quality flags raised).
6. **C6:** Every prediction event in staging carries correct `model_version` matching registry.

Each claim maps to frozen datasets, scripts, and CI jobs.

15.3 Experimental design

- Pre-register metrics and thresholds before final test evaluation.
- Separate tune / validation / test.
- Multiple seeds for stochastic training.
- Ablations: detector-only, tracker-only with frozen detections, fusion on/off.
- Compare against classical baselines (for example HOG-style or motion differencing only where relevant; more importantly GNN vs JPDA on identical detections).

15.4 Statistical humility

Publish confidence intervals where sample sizes allow. For rare swarm events, admit wide uncertainty and rely on twin stress tests plus staged field trials. Do not launder tiny-N results into absolute claims.

15.5 Hardware-in-the-loop

Stage 7 HIL (programme roadmap) must re-run the assurance pack on representative Jetson and sensor hardware. Lab GPU metrics are necessary but insufficient for edge profiles.

15.6 Red-team and peer review

Independent review of evaluation code (can metrics be gamed?), dataset leakage checks, and scenario coverage gaps. Security specialists review adversarial eval scope; systems engineers review latency and degraded-mode behaviour.

15.7 Operator-in-the-loop studies

Where feasible, measure time-to-correct-comprehension and false-alarm burden with frozen model versions. A model that wins IDF1 but doubles nuisance alerts may fail product proof even if ML metrics pass.

15.8 Documentation as evidence

Model cards, data sheets, evaluation reports, and signed promotion records are part of the proof. If it is not written and hashed, it did not happen for audit purposes.

15.9 Pre-ship gate (ML-specific)

Question	Yes/No discipline
Failure modes understood and sliced?	Must be Yes with listed residuals
Observable in prod (metrics, drift, versions)?	Must be Yes
Safe rollback via gateway?	Must be Yes
Complexity proportional to value?	Challenge deep end-to-end if classical meets claims
Want 3-year ownership of this pipeline?	If No, simplify

16. Implementation guide: Counter-Swarm inference-gateway

16.1 Role in the service catalogue

The inference-gateway is the model serving facade: features or frames in; predictions plus version out; canary and rollback supported. Downstream detection, tracking, and fusion services must not load ad-hoc weight files from developer laptops.

16.2 API contract (normative sketch)

Request (conceptual):

- request_id, site_id, sensor_id

- `model_role` (e.g., `eo_detect_v1`, `rf_class_v1`, `appearance_embed_v1`)
- `payload_ref` or inline tensor metadata
- `accepted_model_versions` (optional pin)
- `deadline_ms`

Response:

- `model_version`, `config_version`
- `predictions` (boxes, scores, labels, embeddings—role-specific schema)
- `latency_ms`, degraded flags (thermal, timeout, fallback)
- `calibration_id` if site overlay applied

All prediction events published to the bus must copy `model_version`.

16.3 Deployment topology

Stage 1 (lab): gateway as a Compose service on x86, CPU or GPU optional; synthetic frames from sim-engine.

Later: central gateway for heavy models; side-car or node-local gateway on Jetson for EO with the same request/response schemas. A thin client library prevents schema forks.

16.4 Canary and rollback

- Traffic split by `site_id`, `sensor_id`, or percentage.
- Shadow mode: compute candidate predictions, log diffs, do not affect tracks until approved.
- Automatic rollback triggers: latency SLO breach, NaN rates, precision proxy alarms, crash loops.
- Manual rollback is a first-class operator/SRE action with audit.

16.5 Batching and deadlines

Gateway should batch when it helps GPU utilisation but must respect `deadline_ms`. Prefer dropping or degrading a frame with a flagged miss over delaying the entire bus partition. Document drop policy; tracking must handle measurement gaps.

16.6 Preprocessing ownership

Decide whether adapters or the gateway own resize, colour conversion, and normalisation. Ambiguity here causes train/serve skew—the most common silent accuracy killer. Freeze preprocessing code digests in the model card.

16.7 Integration with detection service

Patterns:

1. **Passthrough:** vendor sensors already emit detections; gateway unused for that modality.

2. **Gateway-first:** raw frames to gateway; detection service consumes predictions and emits detection events.
3. **Hybrid:** edge gateway emits detections; central detection service validates schema and rate-limits.

16.8 Observability

Metrics: QPS, p50/p95/p99 latency, GPU utilisation, batch size, timeout rate, version counters, score histograms, ECE proxies on labeled shadow streams.

Traces: request_id through gateway to detection to track IDs.

Logs: forbid raw image dumps by default; use policy-gated forensic capture.

16.9 Security

- mTLS between services.
- AuthZ on model admin APIs (promote/rollback).
- Signed artifacts only.
- Rate limits on predict endpoints.
- No unconstrained CORS; not a browser-facing API.
- Sanitize any string fields (labels, versions) for log injection.

16.10 Reference sequence (happy path)

1. Registry marks model eo_detect version 1.4.2 as canary.
2. Gateway loads artifact; health check green.
3. Sensor adapter publishes frame ref; detection service calls gateway with role eo_detect_v1.
4. Predictions return with model_version=1.4.2.
5. Tracking associates; track event includes versions.
6. Metrics pipeline compares canary slice to 1.4.1.
7. Promote to stable or rollback.

16.11 Minimal Stage 1 stub behaviour

Even before real GPUs, implement:

- Schema validation.
- Deterministic fake predictions for sim (versioned).
- Latency injection knobs for chaos tests.
- Registry file or service with promote/rollback APIs.
- Mandatory model_version on outputs.

This unblocks tracking/fusion engineering without waiting for final detectors.

16.12 Testing matrix

Test	Intent
Contract tests	Schema compatibility
Golden tensors	Preprocess + infer bit-stability on CPU ref
Latency soak	p95 under load
Canary switch	Version routing correctness
Failure injection	Timeout, OOM, bad artifact
Skew test	Train preprocess vs gateway preprocess

16.13 Ownership boundaries

Concern	Owner
Model training quality	ML Engineering
Gateway service reliability	Software Engineering + ML
Edge hardware profiles	Systems Engineering
Alert budget coupling	Product + ML
Audit of promotions	Security/Governance + ML

17. Non-goals, ethics, and safety boundaries

17.1 Explicit non-goals

This monograph and the Counter-Swarm ML engineering scope do **not** include:

- Weapon guidance algorithms, aim-point generation, or fire-control loops.
- Kinetic effector commands or electronic-attack execution recipes.
- Autonomous engagement without human approval paths.
- Instructions for building spoofers or offensive adversarial attacks.

ML outputs stop at defensive decision support: detections, tracks, fused evidence, calibrated scores, and recommendations that feed human approval services.

17.2 Dual-use awareness

Perception models can be misused. Controls: access control on datasets and weights, purpose limitation in model cards, and legal review for sharing. Technical quality work on detection is still framed as site protection and operator assistance.

17.3 Human authority

No model version may bypass approval services or widen integration API categories. Inference gateway admins must not hold credentials that directly reach effector interfaces in this repository's design.

17.4 Privacy

Cameras may capture incidental persons or vehicles. Retention, redaction, and access policies constrain training data. ML pipelines must enforce dataset retention TTLs and access logs.

18. Open questions, assumptions, and research agenda

18.1 Open questions (stakeholder-closed)

- Air-gapped versus connected sites (affects registry sync).
- Camera and radar vendor list (affects adapters vs learned detect).
- Power/thermal envelope at edge posts.
- Official false-alarm budgets per alert class.
- Legal rules for raw frame retention.

18.2 Research agenda (honest Pass-2 expansions)

1. Formal comparison study: JPDA vs track-oriented MHT vs learned association on Counter-Swarm twin packs with fixed detectors.
2. Range-stratified EO small-object benchmark with public methodology (even if data remain private).
3. Joint calibration of detector scores and filter existence probabilities.
4. Track-to-track fusion with explicit cross-covariance under hybrid edge/central deployment.
5. Operator trust calibration: mapping ECE to human use of confidence glyphs.
6. Continual learning sandbox with contamination alarms.

18.3 Assumptions revisited

If field data show EKF residuals systematically non-Gaussian under agile flight, escalate IMM/particle studies. If Jetson thermal profiles miss SLO, move mid-tier models central and send crops only. If over-merge dominates incidents, prioritise association research over larger detectors.

19. Conclusion

ML engineering for counter-UAS defence is the craft of making perception and tracking **operable**: versioned, calibrated, measurable, and subordinate to human approval. The correct default spine remains classical estimation and data association in the Kalman and Bar-Shalom tradition, with YOLO-style detectors (Redmon et al.) and related deep models supplying measurements where imagery justifies them. MOTChallenge-inspired metrics (MOTA, IDF1; Bernardin and Stiefelwagen; Milan/Leal-Taixé et al.) discipline identity quality. Synthetic twins expand rare regimes but do not replace real holdouts. Jetson edges and central x86 hosts split by bandwidth and fusion need, not by fashion. The Counter-Swarm inference-gateway is the control plane that makes canary, rollback, and audit possible.

Ship the smallest trustworthy slice first. Expand association sophistication when identity metrics demand it. Never confuse a demo reel with an assurance case. Never extend this stack into weapon guidance under the guise of “just ML.”

20. References

Bar-Shalom, Y., & Fortmann, T. E. (1988). *Tracking and Data Association*. Academic Press.

Bar-Shalom, Y., Li, X. R., & Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley. (Commonly cited; imprint listings may show 2001/2002.)

Bernardin, K., & Stiefelwagen, R. (2008). Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008, Article 246309.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (PMLR 70).

Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.

Leal-Taixé, L., Milan, A., Reid, I., Roth, S., & Schindler, K. (2015). MOTChallenge 2015: Towards a benchmark for multi-target tracking. arXiv:1504.01942. (MOTChallenge series; see also subsequent challenge editions.)

Milan, A., Leal-Taixé, L., Reid, I., Roth, S., & Schindler, K. (2016). MOT16: A benchmark for multi-object tracking. arXiv:1603.00831.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv:1804.02767.

Reid, D. B. (1979). An algorithm for tracking multiple targets. *IEEE Transactions on Automatic Control*, 24(6), 843–854.

Additional practitioner context used in programme architecture (not substitutes for the landmarks above): Counter-Swarm Defence internal architecture notes on hybrid edge/central compute, inference–gateway placement, and Stage 0 ML pipeline guidance.

Appendix A: Metric definitions

A.1 Detection

Precision at a threshold λ : fraction of predicted boxes (after matching) that are correct.

Recall at λ : fraction of ground-truth objects matched by a prediction.

F1: harmonic mean of precision and recall at a chosen operating point.

mAP: mean average precision across classes and/or IoU thresholds; report the IoU schedule explicitly (for example COCO-style or a fixed 0.5 IoU for operational simplicity). For C-UAS, always add **range-stratified AP**.

FPR / FNR: false positive and false negative rates at the deployed threshold; pair with alert-budget simulations.

A.2 Tracking (CLEAR / MOTChallenge-style)

FP, FN, IDSW: counts per frame or per timestep after bipartite matching of hypotheses to ground truth.

MOTA: see Section 8.2; can be negative if errors exceed ground-truth counts—report anyway.

MOTP: average localisation precision over matched pairs; secondary to identity metrics for operator trust.

IDF1: identity F1 based on the global allocation of predicted trajectories to ground-truth trajectories (as used in MOTChallenge evaluations).

Fragmentation: number of times a ground-truth track is interrupted in the hypothesis set.

Mostly Tracked / Mostly Lost: fraction of ground-truth trajectories covered for majority / minority of their lifespan (MOTChallenge reporting convention).

A.3 Calibration and ops

ECE: expected calibration error over score bins.

NIS: normalised innovation squared for Kalman diagnostics.

Latency p95: 95th percentile end-to-end infer or track-update time on named hardware.

Throughput: sustained frames or requests per second without deadline violations.

A.4 Fusion-specific

Duplicate track rate: distinct fused IDs per physical object (want near 1).

Modality contribution freshness: age of last contributing measurement per modality.

Cross-sensor ID stability: IDSW measured after fusion, not only intra-modality.

Appendix B: Model card skeleton

```
# Model card: <name>
Author: Victor.I (card owner: <engineer>)
model_version: <semver or hash>
model_role: eo_detect_v1 | rf_class_v1 | ...
status: experimental | canary | stable | retired

## Intended use
Defensive detection/classification for Counter-Swarm operator decision support.
Out of scope: weapon guidance, effector control, offensive spoof design.

## Architecture
Family: YOLOv3-lineage / ... (cite Redmon et al. where applicable)
Input: <shape, colourspace, normalisation>
Output: <boxes, scores, classes>
Preprocess digest: <git sha / hash>

## Training data
dataset_version: <id>
taxonomy_version: <id>
real/synthetic mix: <percentages>
known gaps: <far range, IR crossover, ...>

## Evaluation
packs: <SP-ids>
metrics: mAP, recall@P, ECE, latency on H0/J0
slices: <table>
comparison baselines: <names>

## Hardware
central: <CUDA image digest>
edge: <JetPack, TensorRT engine id>

## Calibration
method: temperature scaling / threshold pack
calibration_id: <id>

## Risks and mitigations
overconfidence, birds FPs, swarm undercount, ...

## Promotion record
approvals: <names/dates>
canary results: <link>
```

Appendix C: Association cost design checklist

1. Are all cost terms documented with units and versions?
2. Is Mahalanobis gating using the same $\mathbf{R}$ as the filter update?
3. Are class incompatibilities hard constraints or soft penalties?

4. Is appearance distance disabled when embedding quality flags are low?
 5. Are coasting tracks penalised enough to avoid stealing measurements from fresh tracks?
 6. Is cluster size capped for JPDA with a defined fallback (GNN inside overflow)?
 7. Are MHT hypothesis caps and N-scan depth recorded in `tracker_config_version`?
 8. Do offline identity-repair tools use a different config ID than online?
 9. Are fusion association costs separate from intra-modality costs?
 10. Do CI tests include a crossing two-target scenario and a cardinality-12 swarm scenario?
-

Appendix D: Extended worked scenarios

D.1 Single quadcopter, day EO, clear sky

Setup: One airframe, central YOLO-style detector, Kalman NCV, GNN association.

Expected: High MOTA, rare IDSW, calibration checked on objectness vs precision.

What goes wrong: Motion blur on pan-tilt zoom cameras; detector score flicker; track death on prolonged blur. Mitigate with coast timeout tuning and temporal class smoothing.

Proof artifacts: SP-day-single metrics table; NIS histogram; latency on Compose GPU.

D.2 Two airframes crossing in front of birds

Setup: JPDA cluster; optional appearance embeddings; secondary bird class.

Expected: IDF1 holds across cross if embeddings or multi-scan hypotheses work; bird measurements mostly rejected.

What goes wrong: GNN identity swap; bird boxes with high objectness steal gates; over-merge after cross.

Proof artifacts: IDSW timeline plots; association entropy traces; ablation GNN vs JPDA vs MHT-lite.

D.3 Radar-led track with intermittent EO confirm

Setup: Radar measurement-level filter; EO detections associated centrally when available; fusion config with sensor bias states.

Expected: Continuous track through EO gaps; EO upgrades class confidence when present.

What goes wrong: Time skew creates side-by-side ghosts; naive track-to-track fusion double-counts.

Proof artifacts: duplicate rate; skew-injected sim; covariance consistency checks.

D.4 High-cardinality twin swarm

Setup: Synthetic N=20; stress JPDA clustering and NMS.

Expected: Documented degradation envelope; not silent collapse.

What goes wrong: NMS under-count; CPU spikes; operator display overload.

Proof artifacts: cardinality error vs N; CPU profiles; alert-budget coupling note.

D.5 Night IR crossover

Setup: Separate IR model registry entry; no EO weight reuse.

Expected: Explicit fail-soft when contrast vanishes; quality flags.

What goes wrong: Team ships EO model on IR stream “temporarily.”

Proof artifacts: modality mismatch monitors; IR-only evaluation pack.

D.6 Jetson thermal soak

Setup: Orin NX engine, outdoor enclosure profile, sustained 30 FPS attempt.

Expected: Either sustained SLO or automatic frame-load shed with degraded flags.

What goes wrong: Lab-only latency sign-off; field throttling; backlog of stale frames.

Proof artifacts: thermal soak test report; drop-policy unit tests.

Appendix E: Train/serve skew catalogue

Skew source	Symptom	Guard
Colour order BGR/RGB	Sudden AP collapse	Golden tensor test
Resize aspect differ	Box bias	Shared preprocess lib
Normalisation mean/std drift	Score shift	Digest pin
NMS in train differs from serve	Density errors	Config version
Letterbox padding mismatch	Systematic offset	Visual QA overlays
Score threshold applied twice	Empty outputs	Contract tests
Augmentations left on in serve	Random FPs	Explicit eval mode flag
INT8 without calibration set	Small-object death	Engine eval gate
Different JPEG quality	Texture shift	Codec in dataset card
Camera ISP auto-exposure	Day/night surprise	Site threshold overlays

Appendix F: Classical parameter starting points (illustrative, not site truth)

These values are **illustrative** for laboratory bring-up against the software simulator. They are not certified field tunings.

- NCV process noise spectral density: start moderate; increase until NIS is not persistently $\gg 1$ on clean single-target truth.
- Measurement $\backslash(R\backslash)$ for EO boxes converted to world coordinates: derive from pixel error at estimated range; inflate at long range.
- $\backslash(P_D\backslash)$: estimate from detector recall at operating threshold on validation; do not use 0.99 wishful defaults.
- Clutter density: measure false positives per gate volume on empty-sky hours.
- Confirmation M-of-N: stricter when FPR high; looser when miss cost high—product decision.
- Coast timeout: bound by manoeuvre capability and sensor revisit; expose to operators as coast age.

Every change requires a config version bump and scenario regression.

Appendix G: Deep detector training recipe checklist

1. Pin dataset and taxonomy versions.
 2. Define range bins and minimum samples per bin per epoch.
 3. Choose input resolution against Jetson and central budgets before long training.
 4. Log class imbalance; apply loss weights carefully (over-weighting rare classes can explode FPR).
 5. Validate augmentations on visual boards weekly.
 6. Early-stop on stratified validation, not training loss.
 7. Export ONNX; build TensorRT; re-evaluate—do not assume parity.
 8. Calibrate scores; freeze threshold packs.
 9. Write model card; register; canary.
 10. Schedule drift monitors before declaring victory.
-

Appendix H: Mapping to Counter-Swarm services

ML concern	Service touchpoint
Frame infer	inference-gateway, detection
Measurement noise	tracking filter config
Association	tracking, fusion
Class soft labels	detection, behaviour, risk
Version audit	all events + audit
Sim labels	sim-engine, offline eval jobs
Operator display trust	operator-api, realtime gateway (quality fields)

ML engineers do not own effector integration. They own the honesty of perception evidence.

Appendix I: Glossary

- **Association:** deciding measurement origin among tracks, clutter, and births.
 - **Canary:** partial release of a model version under observation.
 - **Coast:** predicting a track without measurement updates.
 - **ECE:** expected calibration error.
 - **Gate:** validation region for candidate associations.
 - **IDF1:** identity-aware F1 for multi-object tracking.
 - **IMM:** interacting multiple model filter.
 - **JPDA/JPDF:** joint probabilistic data association (filter).
 - **MHT:** multiple hypothesis tracking.
 - **MOTA:** multiple object tracking accuracy (CLEAR).
 - **NIS/NEES:** normalised innovation/estimation error squared.
 - **PDA/PDAF:** probabilistic data association (filter).
 - **Tracklet:** short trajectory fragment, often stitched offline.
-

Appendix J: Assurance case outline (one page)

Claim: Model stack S is fit for Stage-N defensive trial at site Z for daytime EO cardinality ≤ 5 .

Context: Human approval retained; no effector autonomy; Jetson profile optional.

Argument:

1. Detector evidence: metrics on SP packs (Section 15 claims C1, C4).
2. Tracker evidence: IDF1/IDSW on same detections (C2).
3. Calibration evidence: ECE and threshold packs (C3).

4. Ops evidence: gateway rollback drill success (C6).
5. Residual risks: listed slices with mitigations and monitoring.

Defeaters: domain shift after camera change; swarm $N > 5$; IR night (out of claim scope).

Evidence store: hashed evaluation report URIs in registry.

Appendix K: Expansion notes for Pass 2 (integrity-preserving)

To grow this monograph toward longer dissertation scale without fabricating citations:

- Add full derivations of PDA weights and JPDA marginalisation with worked numeric toy examples.
- Reproduce CLEAR MOT matching algorithm step-by-step on a 5-frame toy sequence.
- Add related-work survey tables for learned MOT (Tracktor, CenterTrack, ByteTrack, etc.) with honest latency/identity comparisons on Counter-Swarm packs once data exist.
- Add extended sensor-fusion mathematics for track-to-track fusion with cross-covariance.
- Add annotated failure casebook from sim and field trials as they accrue.
- Add appendices of configuration schemas once Stage 1 code lands.

Pass 2 must remain citation-honest: new claims require new real sources or clearly labeled internal empirical reports.

Document control

Field	Value
Title	ML Engineering for Detection, Classification, Tracking, and Fusion in Counter-UAS Defensive Systems
Author	Victor.I
Programme	Counter-Swarm Defence research monograph series
Path	docs/research/ml-engineering/detection-tracking-fusion-monograph.md
Defensive scope	Perception and decision support only; no weapon guidance
Citation policy	Landmark works only; no fabricated DOIs

Author: Victor.I

Appendix L: Extended technical deep dive — filters, association mathematics, and evaluation protocols

This appendix expands Pass-1 material with additional engineering depth suitable for dissertation-scale ownership. It remains deliberately free of weapon guidance content.

L.1 Innovation statistics and gate design

After prediction, the innovation $\nu = z - H\hat{x}$ and innovation covariance $S = HPH^T + R$ define the normalised distance $d^2 = \nu^T S^{-1} \nu$. Under correct linear-Gaussian assumptions, d^2 is chi-squared with dimension equal to the measurement rank. Gates are level sets of d^2 (or ellipsoidal volumes). Practical notes:

- Underestimated R shrinks gates and raises missed associations (FN tracks / fragmentation).
- Overestimated R admits clutter and raises IDSW / ghosts.
- Time-varying R from detector localisation-vs-score curves should be monotonic and unit-tested.

Gate probability P_G (probability that the true measurement falls in the gate when detected) couples to PDA/JPDA weight formulae in Bar-Shalom and Fortmann (1988). If you change P_G without updating association code constants, you have introduced a silent logic bug.

L.2 PDA weight structure (conceptual)

For a single track with gated measurements $\{z_i\}_{i=1}^m$, PDA forms association probabilities β_i for each measurement and β_o for the “no detection / all clutter” event. The update is a mixture:

$$\begin{aligned} &[\\ \hat{x}^+ &= \sum_{i=0}^m \beta_i \hat{x}_i \\ &] \end{aligned}$$

with a covariance that includes a spread-of-means term. Engineers must implement the spread term; omitting it yields optimistic covariances and over-tight subsequent gates—exactly the path into over-merge.

Clutter intensity λ (or spatial density) is the sensitive parameter. Estimate it from:

1. Empty-sky measurement rates per unit gate volume, or
2. Detector false positive rate times gate volume under the current threshold.

Re-estimate after every detector threshold change.

L.3 JPDA joint events (conceptual)

For a cluster with tracks $\{T\}$ and measurements $\{M\}$, joint association events assign at most one measurement per track and at most one track per measurement, with leftover measurements as clutter and leftover tracks as missed detections. Exact enumeration is factorial in cluster size; practical systems:

- Cluster by shared gate overlap graphs.
- Cap cluster size; split with heuristics when over cap.
- Use efficient approximate JPDA variants when needed, but freeze the variant ID in `tracker_config_version`.

Marginal β_j for track j and measurement i are sums over joint events. Soft updates again require spread-of-means covariance corrections.

Coalescence: when two targets remain inside each other's gates, JPDA posterior means can collapse toward each other. Monitoring inter-track distance versus combined covariance eigenvalues detects coalescence risk. Responses: appearance features, hard mutual exclusion for one scan with MHT backup, or temporary track freeze with operator-visible ambiguity flags.

L.4 MHT scoring intuition

Hypothesis scores typically accumulate log-likelihood ratios for association choices, missed detections, and clutter. N-scan pruning retains only branches that differ in the last N association decisions. Track-oriented MHT structures hypotheses around tracks rather than global scene trees to control memory. Regardless of variant:

- Bound maximum hypotheses per cluster.
- Log prune reasons at debug level for incident review.
- Never present the full tree to operators; present primary hypothesis plus ambiguity indicators.

L.5 IMM switching for agile UAS

Interacting multiple model filters maintain a bank of Kalman/EKF filters with different motion models and mix state estimates using mode probabilities. Mode transition matrices encode how often modes switch. For small UAS:

- Mode 1: nearly constant velocity cruise.
- Mode 2: higher process noise manoeuvre.
- Optional Mode 3: hover / near-zero velocity with different measurement expectations for Doppler-capable radar.

Mode probability entropy is a useful track quality feature: high entropy means the filter is unsure which dynamics apply, often near turns—association should be more conservative (larger effective gates but lower trust in identity-critical merges).

L.6 From image detections to world-frame measurements

EO boxes live in pixels. Tracking in pixels is acceptable for single-camera short horizons; multi-sensor fusion usually needs a world or site frame. Pipeline:

1. Undistort using camera intrinsics version $\backslash(C_v\backslash)$.
2. Ray-cast through camera pose $\backslash(P_v\backslash)$ (extrinsics).
3. Intersect with altitude hypothesis (barometric prior, radar height, or ground plane assumption)—**altitude error dominates ground-range error**.
4. Convert to measurement $\backslash(z\backslash)$ with $\backslash(R\backslash)$ inflated for pose uncertainty and box jitter.

Version $\backslash(C_v\backslash)$ and $\backslash(P_v\backslash)$ in the same governance system as neural models. A silent extrinsics drift after mast maintenance is a fusion incident, not a “detector got worse” incident.

L.7 Evaluation protocol: frozen detection challenge

To compare trackers fairly (MOTChallenge methodological lesson):

1. Freeze a detection file per sequence (or live detector version pin).
2. Run tracker candidates against identical detections.
3. Report IDF1, IDSW, MOTA, CPU time.
4. Then allow each tracker a joint tune with its preferred detector only in a separate “system” leaderboard.

Without this split, detector upgrades masquerade as tracker wins.

L.8 Matching thresholds for aerial objects

Pedestrian MOT often uses IoU 0.5 in image space. Aerial small objects have tiny boxes; IoU matching is unstable. Prefer:

- Centre distance normalised by $\max(\text{box scale, minimum pixel tolerance})$, or
- World-frame Euclidean gates with range-dependent thresholds.

Document the matcher in the evaluation harness hash. Changing matcher parameters retrospectively to improve a favoured model is scientific misconduct in an assurance context.

L.9 Bootstrapped confidence intervals

For primary metrics on finite scenario packs, bootstrap sequences (resample scenarios or trajectory IDs) to report intervals. If intervals for model A and B overlap heavily, do not declare victory in promotion text.

L.10 Continuous evaluation in production

Labeled truth is rare in production. Proxies:

- Multi-sensor agreement rates (EO confirms radar) as a soft precision proxy.
- Orphan detection rates.

- Track birth/death churn.
- Score histogram shifts.
- NIS distribution shifts.

Proxies trigger investigation; they do not automatically retrain. Human-labeled incident clips feed the offline assurance packs.

L.11 Dataset documentation sheet (companion to model card)

Every dataset version should answer:

- Who collected and under what consent/legal basis?
- Geographic and temporal coverage?
- Sensor serials and firmware?
- Annotation vendor and guide version?
- Known systematic biases (only small consumer quads, only midday, etc.)?
- Forbidden uses?

L.12 Synthetic fault injection library (defensive eval)

The sim-engine should inject faults that ML evaluation consumes:

- Increased clutter rate.
- Dropped frames.
- Bias steps in extrinsics.
- Clock skew.
- Class-confusable objects (birds).
- Thermal contrast collapse (IR).

Each fault has an ID referenced in metric tables (“SP-o4 + FAULT_CLOCK_SKEW_40ms”).

L.13 Quantisation validation protocol

1. Evaluate FP32 reference on pack SP.
2. Build INT8 engine with calibration set Cal (not test).
3. Evaluate INT8 on SP; diff per range bin.
4. If far-range recall drop exceeds budget, refuse edge promotion; consider mixed precision or higher resolution central crops.
5. Recalibrate scores on INT8 outputs separately—do not reuse FP32 temperature parameters blindly.

L.14 Feature stores versus frame stores

Behaviour and risk may consume track features. Do not casually dump raw video into a feature store. Define:

- Hot track features on the bus/DB.
- Cold frames in object storage with TTL.

- Training extract jobs that materialise chips with hashes.

This keeps privacy and cost under control.

L.15 On end-to-end learned trackers

End-to-end models that map video to tracks can look strong on academic MOT. For Counter-Swarm Stage 1–4, treat them as research challengers because:

- Assurance modularity suffers (harder to swap detectors).
- Latency and memory on Jetson may not fit.
- Failure explanation for operators is weaker.
- Fusion across radar/RF is unnatural.

Revisit when modular baselines plateau and HIL evidence exists.

L.16 Interface contracts for appearance embeddings

Embedding vectors must declare:

- Dimension and distance metric.
- Whether L2-normalised.
- Quality flag (occlusion, blur).
- Model version.

Association costs should ignore embeddings when quality is low rather than trusting garbage cosine values.

L.17 Multi-camera handoff

When multiple EO posts cover overlapping volumes:

- Prefer fusion in world frame over ID handoff heuristics alone.
- Use appearance + motion for re-acquire after blind zones.
- Test with deliberate camera dropouts.

Handoff bugs present as fragmentation even when single-camera MOTA is fine.

L.18 Latency budgets and waterfall

Example waterfall (illustrative):

Stage	Budget share
Capture + adapter	10%
Infer (gateway)	40%
Track update	20%
Fusion	15%
Bus + UI	15%

ML owns infer and influences track/fusion cost through measurement rates. A detector that emits three duplicates per object burns association CPU and is a latency defect, not just an NMS aesthetic issue.

L.19 Incident postmortem template (ML)

1. Symptom (operator narrative + track IDs).
2. Model and config versions live at the time.
3. Scenario reproduction (sim or replay).
4. Root cause class (detector, association, fusion, calibration, clock, UI).
5. Metric gaps that failed to catch it.
6. Corrective action + regression test ID.
7. Communication to ops (what to distrust meanwhile).

L.20 Why “more data” is not always the fix

If association coalescence dominates, more EO labels will not help. If extrinsics drift, more training will not help. If alert budgets are saturated, higher recall may harm operations. Diagnosis before collection spend is part of ML engineering maturity.

L.21 Relationship to risk models

Risk engines should consume calibrated probabilities and track quality, not raw logits. ML must provide monotonic transforms and refuse to ship uncalibrated scores into severity mapping. If risk policy changes thresholds, ML recalibrates operating points and re-runs alert-budget simulations.

L.22 Software testing patterns for numerical trackers

- Property tests: covariance positive definite after updates.
- Golden trajectories: fixed measurements, expected state within tolerance.
- Fuzz: random clutter fields, ensure no crashes and bounded CPU.
- Metamorphic: translating all measurements by a constant should translate state (within frame rules).

L.23 Documentation debt that becomes safety debt

Missing model cards, unpinned datasets, and undocumented NMS thresholds are not paperwork failures; they are inability to roll back safely. Treat docs as runtime dependencies of the gateway promotion API.

L.24 Ethical evaluation reporting

When publishing internal scores, avoid claiming human-equivalent situational awareness. Report miss rates honestly. Do not hide bird FPR in footnotes. Defensive systems earn trust through calibrated humility.

L.25 Summary of appendix stance

Filters and association are not legacy embarrassments to be replaced at the first opportunity by an undifferentiable monolith. They are the language in which Counter-Swarm can state claims, gates, and failures. Deep learning extends that language at the measurement boundary. The gateway enforces versioned speech. Evaluation protocols—CLEAR MOT, MOTChallenge discipline, calibration, drift—decide whether anyone should listen.

Appendix M: Annotated reading path for new ML engineers on the programme

Week 1: Kalman (1960) for intuition; implement a 2D NCV filter on sim truth; pass NIS checks.

Week 2: Bar-Shalom and Fortmann (1988) chapters on gating and PDA; implement GNN and PDA; compare IDSW on crossing scenario.

Week 3: JPDA cluster toy problem; read Bar-Shalom, Li, and Kirubarajan (2001) sections used by the team for IMM/navigation-style estimation practice.

Week 4: Redmon et al. YOLO papers; train a small detector on twin chips; measure range-stratified AP.

Week 5: Bernardin and Stiefelhagen (2008); Milan et al. MOT16 / MOTChallenge protocols; wire evaluation CI.

Week 6: Guo et al. (2017) calibration; add ECE to pipeline; temperature scaling.

Week 7: Package through inference-gateway stub; canary drill; break preprocessing on purpose and catch with golden tensors.

Week 8: Jetson profile bring-up (when hardware available); TensorRT parity tests; thermal soak.

This path emphasises ownership competence over survey breadth.

Appendix N: Configuration sketch (illustrative YAML)

```
# Author: Victor.I
# Illustrative only – not a live production config
tracker_config_version: track.kalman.jpda.v3
filter:
  type: imm
  models: [ncv, manoeuvre]
  pd: 0.85
  gate_pg: 0.99
association:
  type: jpda
  max_cluster_size: 8
  overflow_fallback: gnn
  appearance:
    enabled: true
    max_distance: 0.35
    ignore_if_quality_below: 0.4
costs:
  mahalanobis_weight: 1.0
  class_mismatch_penalty: 4.0
  coast_age_penalty_per_second: 0.1
lifecycle:
  confirm_m_of_n: [3, 5]
  coast_timeout_s: 2.5
  delete_timeout_s: 5.0
```

Any field change requires a new `tracker_config_version` and regression pack run.

Appendix O: Counter-arguments and rebuttals

“Just use a large vision model end-to-end.”

Rebuttal: latency, assurance modularity, multi-sensor fusion, and audit requirements dominate demo accuracy. Large models may assist offline review later.

“MHT is too old.”

Rebuttal: identity under ambiguity is still the hard problem; modern learned methods often reintroduce deferred decisions under new names. Benchmark on IDF1 before discarding.

“Synthetic data is enough.”

Rebuttal: domain gap in ISP, clutter, and weather repeatedly kills field metrics; real holdouts are mandatory ship gates.

“Calibration can wait.”

Rebuttal: uncalibrated scores corrupt risk and operator trust; ECE is cheaper than incident-driven rewrites.

“Edge everything on Jetson clusters.”

Rebuttal: fusion, audit, and operator systems want central ops maturity; hybrid remains the programme baseline.

“Pi is cheaper for cameras.”

Rebuttal: cheaper hardware that misses recall SLO is expensive in operational risk; Pi remains light-adaptor only.